

Compilador Pascal

Projeto 2 VA — Análise Semântica

Disciplina: Compiladores — Professor: Waldemar Pires Ferreira Neto

1. Descrição do Projeto

Este projeto implementa um compilador para um subconjunto da linguagem Pascal, com ênfase na **análise semântica**. O compilador realiza as seguintes fases:

- Análise Léxica (Flex) — separa o código em tokens
- Análise Sintática (Bison) — verifica a estrutura gramatical
- Análise Semântica — verifica compatibilidade de tipos em atribuições e expressões
- Tabela de Símbolos — armazena informações de variáveis e procedimentos

2. Arquivos do Projeto

Arquivo	Descrição
lex.l	Analizador léxico (Flex)
parser.y	Analizador sintático + semântico (Bison)
symbol_table.c/h	Tabela de símbolos
type_checker.c/h	Verificador de tipos (checagem semântica)
code_gen.c/h	Gerador de código intermediário
Makefile	Script de compilação (make)
entrada.pas	Exemplo de programa Pascal válido
teste_erro.pas	Exemplo com erro semântico para teste

3. Verificações Semânticas Implementadas

O projeto realiza **6 verificações semânticas** durante a análise:

#	Verificação	Exemplo de Erro
1	Tipo incompatível na atribuição	integer := real
2	Condição não-booleana no if	if 1 then ...
3	Condição não-booleana no while	while 1 do ...
4	Operação aritmética com booleano	true + 1
5	Tipos incompatíveis em comparação	1 > true
6	Variável não declarada	x := 1 sem var x

4. Como Compilar e Executar

Pré-requisitos (instalados via Scoop):

- **winflexbison** (flex.exe + bison.exe) — geradores de analisadores
- **gcc** (GCC 15.2.0) — compilador C
- **make** (opcional) — automatiza a compilação
- Instalação: *scoop install winflexbison gcc make*

Método 1 — Passo a passo (sem make):

```
# 1. Gerar o parser (Bison)
bison -d -v parser.y

# 2. Gerar o lexer (Flex)
flex lex.l

# 3. Compilar cada módulo
gcc -c lex.yy.c -I. -include symbol_table.h -o lexer.o
gcc -c parser.tab.c -I. -o parser.o
gcc -c symbol_table.c -I. -o symbol_table.o
gcc -c type_checker.c -I. -o type_checker.o
gcc -c code_gen.c -I. -o code_gen.o

# 4. Linkar tudo
gcc *.o -o compilador.exe

# 5. Executar
.\compilador.exe entrada.pas
```

Método 2 — Usando make (recomendado):

```
make all          # compila tudo
make run          # compila e executa com entrada.pas
make clean        # limpa arquivos gerados
```

5. Exemplos de Execução

5.1. Código válido (sem erros)

```
program exemplo;
var
  x, y : integer;
  z : real;
begin
  x := 1;
  y := 2;
  z := 3.5
end.
```

Saída esperada:

```
=== Compilador Pascal ===
Verificação de tipos: OK

=== Tabela de Símbolos ===
Nome      Tipo      Categoria  linha:
exemplo   procedure procedure  linha: 1
x         integer  variable  linha: 3
y         integer  variable  linha: 3
z         real    variable  linha: 4
Total: 4 símbolos
```

5.2. Código com erro semântico

```
program testeerro;
var
  x : integer;
  y : real;
begin
  x := 10;
  y := 3.14;
```

```
    x := y    # ERRO: integer := real
end.
```

Saída esperada:

```
=== Compilador Pascal ===

=== Erros de Tipo (1) ===
Linha 9: Tipos incompatíveis na atribuição

=== Tabela de Símbolos ===
Nome      Tipo      Categoria  linha:
testeerro procedure procedure  linha: 1
x          integer   variable   linha: 3
y          real     variable   linha: 4
Total: 3 símbolos
```

6. Observações

- Aceita coerção implícita de integer para real (ex: realVar := 10)
- **Rejeita** coerção implícita de real para integer (ex: intVar := 3.5)
- If/While exigem condição booleana
- Warnings de conflitos (shift/reduce) são normais e não afetam o funcionamento
- Ferramentas instaladas via **Scoop** (winflexbison, gcc, make)

Instrução para o professor: descompacte o ZIP, abra o PowerShell na pasta. Digite *make all* para compilar, depois *.\compilador.exe entrada.pas* (válido) e *.\compilador.exe teste_erro.pas* (erro semântico).